



TÓPICOS EM INFORMÁTICA

Gerenciamento de memória

MEMÓRIA

- Jogos e aplicativos construídos em cima de interfaces gráficas como o opengl demandam a utilização de muitos recursos.
- Imagens (textura), sons, músicas, modelos 3d, skins, tudo isso pode consumir um nível alto de memória RAM da CPU e da GPU.
- Contudo, é impossível deixar o garbage collector do Java funcionar quando falamos de GPU. Se você construir um sprite que usa uma textura (alocada na GPU) e essa textura for apagada, a aplicação irá renderizar um bloco preto onde deveria estar a textura.

MEMÓRIA

- Felizmente, a libgdx fornece uma série de classes para facilitar o gerenciamento de memória e você não ter que fazer isso na mão.
- Caso usasse o OpenGL, por exemplo, todos os disposes e carregamento de texturas seriam feitos manualmente.
- Pra economizar memória (principalmente quando tratamos de dispositivos mobile), os recursos devem ser apagados assim que eles não forem mais utilizados.



MEMÓRIA

- Se estiver na dúvida se você deve “apagar”/“liberar” o conteúdo da memória, cheque se o objeto contém um método chamado `dispose()`.
- Caso o objeto contenha esse método ele deve sim ser apagado (chamando exatamente a função `dispose`) **a partir do momento em que ele não estiver mais sendo utilizado.**
- **Se não forem apagados**, com o tempo, a sua aplicação ficará cada vez mais lenta pois estará utilizando cada vez mais memória.



MEMÓRIA

- Chequem, por exemplo, se a classe Texture contém o método dispose.
- O que acontecerá se chamarmos um método dispose de um objeto?
- O objeto Java será apagado?



OBJECT POOLING

- Existe um conceito chamado **object pooling, i.e., reutilização de objetos**. Por exemplo, em jogos em que existe frequente recriação de projéteis ou elementos gráficos, essa técnica pode ser interessante.
- Ela consiste de criar um conjunto de objetos (object pool) para evitar a **recriação de novos objetos o tempo todo**.
- Quando você precisa de um novo objeto você checa o pool, se **houver um objeto livre, ele é retornado**. Se a pool estiver vazia um **novo objeto é criado** e retornado. Quando ele não for mais utilizado ele volta pra pool (ao invés de dar dispose).



OBJECT POOLING

- A libgdx fornece algumas ferramentas para object pooling:
 - **Poolable** interface
 - **Pool**
 - **Pools**
- Implementar **Poolable** significa dizer que o seu objeto terá um método **reset()** que será chamado quando você liberar o objeto.

OBJECT POOLING

```
/**
 * Exemplo de classe para uma bala/tiro.
 */
public class Bullet implements Pool.Poolable {

    public Vector2 position;
    public boolean alive;

    /**
     * Construtor de uma bala/tiro.
     */
    public Bullet() {
        this.position = new Vector2();
        this.alive = false;
    }

    /**
     * Inicializa a bala. Esse método deve ser chamado depois de
     * pegar uma bala da pool de balas.
     */
    public void init(float posX, float posY) {
        position.set(posX, posY);
        alive = true;
    }
}
```

```
/**
 * Método que é chamado quando esse objeto é
 * Liberado. É chamado automaticamente quando retorna para
 * o Pool.
 */
@Override
public void reset() {
    position.set(0,0);
    alive = false;
}

/**
 * Método chamado em cada frame (no método render)
 * que atualiza a posição da bala.
 */
public void update (float delta) {
    // se estiver fora da tela então para de
    atualizar
    if (isOutOfScreen()) {
        alive = false;
    } else {
        //posição da bala
        position.add(1*delta*60, 1*delta*60);
    }
}
}
```




OBJECT POOLING

```
public class World {  
  
    // array contendo as balas ativas.  
    private final ArrayList<Bullet> activeBullets = new ArrayList<Bullet>();  
  
    // pool de balas.  
    private final Pool<Bullet> bulletPool = new Pool<Bullet>() {  
        @Override  
        protected Bullet newObject() {  
            return new Bullet();  
        }  
    };  
  
    public void update(float delta) {  
  
        // se quiser usar uma nova bala:  
        Bullet item = bulletPool.obtain();  
        item.init(2, 2);  
        activeBullets.add(item);  
  
        // se quiser liberar balas "mortas", retornando elas para o pool  
        Bullet item;  
        final int len = activeBullets.size;  
        for (int i = len; --i >= 0;) {  
            item = activeBullets.get(i);  
            if (item.alive == false) {  
                activeBullets.removeIndex(i);  
                bulletPool.free(item);  
            }  
        }  
    }  
}
```

EXERCÍCIO

- Faça um personagem que atire diversas balas ao clique de um botão usando o `InputProcessor`.
- Altere a proposta anterior `adicionando o object pooling` aos tiros (bullets).
 - Quando o projétil sair da tela ele deve ser “liberado” para o pool. Ele não estaria mais “alive”.



ASSET MANAGER

- Você pode e deve carregar Textures, TextureAtlas, Sounds, BitmapFonts, etc com o uso da classe **AssetManager**.
- AssetManager te ajuda a **carregar e gerenciar** os seus assets (recursos):
 - O carregamento da maioria dos recursos é feita de **forma assíncrona**. Isto é, enquanto seu jogo carrega os modelos 3d, texturas, sons, etc, ele pode mostrar uma **tela de espera (loading)**.
 - Os assets tem uma **contagem de referência**. Se dois assets ou objetos A e B dependem do asset C, C não será disposed até que A e B sejam disposed.
 - **Mais importante:** se você carregar **duas vezes o mesmo asset**, ele será carregado **apenas uma vez**, **ocupando a memória apenas uma vez**.



ASSET MANAGER

- Um novo asset manager é instanciado da seguinte forma:

```
AssetManager manager = new AssetManager();
```

- Para carregar assets, o AssetManager precisa saber como **carregar um asset específico**. Essa funcionalidade é implementada através de loaders específicos.
- **Exemplo:** Pixmaps são carregados através de PixmapLoader, Textures são carregados através de TextureLoader. BitmapFonts são carregadas através de BitmapFontLoader, etc.



ASSET MANAGER

- Carregar um asset é simples:

```
manager.load("data/mytexture.png", Texture.class);  
manager.load("data/myfont.fnt", BitmapFont.class);  
manager.load("data/mymusic.ogg", Music.class);
```

- Essas chamadas colocam os assets a serem carregados **na fila**. Eles serão carregados na ordem em que são chamados.
- Para de fato efetuar o carregamento dos assets precisamos **chamar o método `update` de forma contínua** (próximo slide).



ASSET MANAGER

```
public MyAppListener implements ApplicationListener {  
  
public void render() {  
    if(manager.update()) {  
        //quando terminar de carregar tudo update retornará true!  
        //a partir desse momento podemos incluir um código aqui para  
mudar de tela, por exemplo.  
    }  
  
    // enquanto carregamos podemos pegar o progresso de  
carregamento da seguinte forma:  
    float progress = manager.getProgress()  
    // é possível printar o progresso na tela com uma BitmapFont  
}
```



ASSET MANAGER

- Caso você queira **travar a tela (isso inclui parar de mostrar o progresso do loading na tela)** e chamar todos os assets de uma vez você pode simplesmente chamar a seguinte função:

```
manager.finishLoading();
```

- Isso irá bloquear a thread da sua aplicação **até que todos os assets de fato tenham sido carregados.**



OBTENDO OS ASSETS

- Uma vez carregados, os assets podem ser obtidos da seguinte forma:

```
Texture tex = manager.get("data/mytexture.png", Texture.class);  
BitmapFont font = manager.get("data/myfont.fnt", BitmapFont.class);
```

- Também é possível verificar se o asset já foi carregado:

```
if(manager.isLoaded("data/mytexture.png")) {  
    // a textura já foi carregada  
    Texture tex = manager.get("data/mytexture.png", Texture.class);  
}
```




DISPOSE

- Caso esteja usando o AssetManager, a forma mais interessante de fazer o dispose dos recursos é através da função unload:

```
manager.unload("data/myfont.fnt");
```

- Nesse caso o asset manager vai ver **se não existe outro objeto referenciando** esse e, caso não exista, fará o dispose no recurso.

- Caso queira apagar todos os assets de uma vez é possível fazer:

```
manager.clear();
```

//ou

```
manager.dispose();
```



EXERCÍCIO

- Troque a **forma de carregar texturas** no código que vocês fizeram até agora para o AssetManager.